// BLUEPRINT IMPLEMENTATION GUIDE

PROJECT ANOMALY — Follow-Along Setup

Node-level Blueprint wiring, Blackboard setup, Behavior Tree assembly, and AI configuration for every system. Written for developers learning UE5 on the fly.

UE 5.4 | BLUEPRINT ONLY | FLOWCHART NODES | BEGINNER FRIENDLY

// 01 UE5 CONCEPTS PRIMER

> A quick reference for every UE5 concept used in this guide. Skip ahead if you already know these.

CONCEPT Execution Pins (white arrows) vs Data Pins (coloured circles)

White arrow pins control order of execution — think of them as wires that say "now do this, then do that". The left arrow is "In" (trigger me), the right is "Out" (what runs next).

Coloured circle pins carry data. **Red = Bool** · **Blue = Int** · **Green = Float** · **Pink = String/Text** · **Teal = Object ref** · **Yellow = Vector** · **Orange = Delegate/Event**. You can only connect matching types.

CONCEPT ActorComponent (AC_), Event Dispatchers, Interfaces (BPI_)

ActorComponent — reusable logic chunk attached to an Actor. Access via `Get [ComponentName]` → drag off the teal pin → call functions.

Event Dispatcher — a broadcast button. One component calls it; any Blueprint that has bound a Custom Event to it reacts independently. This is how systems stay decoupled.

Blueprint Interface — a contract. Add it via Class Settings → Interfaces → Add. Call it on any actor — if that actor implements it the call goes through; if not, nothing crashes. Always the preferred way to talk between systems.

CONCEPT How to read the flowcharts in this guide

Each node is shown as a coloured Blueprint node card. Arrows between cards show execution flow. Labels on connectors show which pin connects. Branch splits show True/False paths side by side. A **teal annotation** beside a node explains why it exists.

PROJECT & FOLDER SETUP

> Do this once before making any Blueprints. Getting the structure right now prevents painful reorganisation later.

- **New project:** First Person template, Blueprint only, no Starter Content. Name it `ProjectAnomaly`.
- **Rename content folder:** `Content/FirstPerson` → `Content/ProjectAnomaly`. Delete the default FirstPersonCharacter BP.
- **Create subfolders:** `Core/GameModes`, `Core/GameInstances`, `Core/SaveGame`, `Player/Character`, `Player/Components`, `Player/Camera`, `Player/Animation`, `Weapons/Base`, `Skills/Base`, `Enemies/Base`, `Enemies/Brute`, `Enemies/FlyingEye`, `Enemies/Spawner`, `Enemies/Fodder`, `Enemies/Shared_AI`, `Combat`, `Arena`, `UI`, `Interfaces`, `Data`, `Structs`.
- **Game Mode:** Edit → Project Settings → Maps & Modes → set Default GameMode to `BP_GM_Anomaly`.
- **Enable AI:** Edit → Plugins → confirm **AIModule** and **GameplayTasks** are enabled.
- **NavMesh:** Place Actors panel → drag **Nav Mesh Bounds Volume** into your test level → scale to cover the floor → press **P** to see it turn green.

FS_DAMAGEINFO STRUCT

Create this struct first — it is used by every single damage call in the game. Right-click in `Structs/` → Blueprint → Structure → name `FS_DamageInfo`.

CREATE Struct members

MEMBER	TYPE	PURPOSE
BaseDamage	Float	Raw damage amount before multipliers
bIsWeakPointHit	Bool	Triggers rage generation and WP multiplier
bIsArmourDamage	Bool	Routes to armour pool instead of health
bIsNextShotBoosted	Bool	Rage 300% multiplier flag
Instigator	Actor Object Ref	Who caused the damage
HitLocation	Vector	World position for VFX spawning

WHY THIS STRUCT EXISTS

1. **Single entry point:** Every damage call in the game passes one FS_DamageInfo. No overloaded functions, no separate armour-damage vs health-damage functions.
2. **Self-routing:** The receiver reads bIsArmourDamage and bIsWeakPointHit to decide what to do — the caller doesn't need to know the enemy's internal structure.
3. **Future-safe:** Adding a new damage type (e.g. elemental, explosive) means adding a Bool to this struct — nothing else changes.

🔧 DEBUG & TEST — FS_DAMAGEINFO

HOW TO TEST

- **Print struct contents:** In any BP that receives FS_DamageInfo, drag off the struct pin → `Break FS_DamageInfo` → connect each output to a `Print String` node. Hit play and fire at an enemy to see all field values in the viewport.
- **Verify bIsWeakPointHit:** Add a `Branch` off the break — True path prints "WP HIT" in green. Confirm it only appears when your shot lands on a BP_WeakPointActor collision sphere.
- **Armour routing check:** Set a Brute's MaxArmour to 999 in DT_Enemies and fire at it. CurrentHealth should not change. Break the DamageInfo inside AC_EnemyHealthComponent and confirm bIsArmourDamage=true for armour-targeted shots.

EXPECTED RESULTS

- ✓ PASS — Pistol shot on enemy body: BaseDamage > 0, bIsWeakPointHit=false, bIsArmourDamage=false
- ✓ PASS — Pistol shot on WP sphere: bIsWeakPointHit=true, rage meter increases
- ✓ PASS — AR shot on Brute: bIsArmourDamage=true, armour decreases, health unchanged
- ✗ FAIL — If health drops when armour > 0: check the bIsArmourDamage flag is being set correctly in the weapon's DamageInfo struct construction

BLUEPRINT INTERFACES

> Create all interfaces before wiring any systems. In `Interfaces/` → right-click → Blueprint → Blueprint Interface. For each function, use the Details panel to add Inputs and Outputs.

BPI_DAMAGEABLE — Implemented by: Player, all enemies

FUNCTION	INPUTS	OUTPUTS
ReceiveDamage	DamageInfo (FS_DamageInfo)	—
IsAlive	—	Bool
GetCurrentHealth	—	Float

BPI_RAGEABLE — Implemented by: BP_PlayerCharacter only

FUNCTION	INPUTS	OUTPUTS
IsEnraged	—	Bool
GetRagePercent	—	Float (0–1)
NotifyWeakPointHit	RageAmount (Float)	—

BPI_PARRYABLE — Implemented by: all enemy projectiles

FUNCTION	INPUTS	OUTPUTS
CanBeParried	—	Bool
OnParried	ParryingPawn (Pawn ref)	—
OnPerfectParried	ParryingPawn (Pawn ref)	—

BPI_INTERACTABLE + BPI_SKILLEQUIPPABLE — Save points and skills

INTERFACE	FUNCTION	INPUTS / OUTPUTS
BPI_Interactable	Interact	In: Pawn ref
BPI_Interactable	CanInteract	In: Pawn → Out: Bool
BPI_SkillEquippable	Execute	In: Pawn ref (instigator)
BPI_SkillEquippable	GetSkillData	Out: DA_SkillData ref

📌 **To implement an interface on a Blueprint:** Open the BP → click **Class Settings** → Details panel → Implemented Interfaces → Add → search your BPI. Interface functions appear in My Blueprint → Interfaces. Double-click one to implement. Functions with return values need a **Return Node** at the end.

🔧 DEBUG & TEST — BLUEPRINT INTERFACES

HOW TO TEST

- **Test BPI_Damageable:** In the level, select any enemy → right-click graph → `Call Function on Actor` → search ReceiveDamage → wire a struct with BaseDamage=50. Hit play and trigger it manually. Enemy health should drop by 50.
- **Test interface on non-implementor:** Call ReceiveDamage on a plain static mesh actor. The game should not crash — the call silently does nothing. This verifies interface safety.
- **Test BPI_Rageable.IsEnraged:** Add a `Print String` that calls IsEnraged on the player every second via a repeating timer. Fill the rage meter and confirm the print flips to True.

EXPECTED RESULTS

- ✓ PASS — ReceiveDamage on enemy: health decreases by 50
- ✓ PASS — ReceiveDamage on plain actor: no crash, no effect
- ✓ PASS — IsEnraged prints False normally, True during enrage
- ✗ FAIL — "does not implement interface" error: Class Settings → Implemented Interfaces must list the BPI. Check this on the target Blueprint.

AC_HEALTHCOMPONENT

> Single source of truth for player HP. Create in `Player/Components/` → Blueprint Class → parent = **ActorComponent**.

VARIABLES Add these in My Blueprint → Variables

NAME	TYPE	DEFAULT	EDITABLE
MaxHealth	Float	100.0	✓
CurrentHealth	Float	100.0	✗
bIsInvincible	Bool	false	✗

Add 2 Event Dispatchers: `OnDamageTaken` (input: DamageAmount Float) and `OnDeath` (no inputs).

GRAPH Event BeginPlay — initialise health

EVENT BEGINPLAY

Event BeginPlay
- Exec Out

Entry point — runs once when the component is created

SET CurrentHealth
- Exec In
- Value ← MaxHealth
- Exec Out

Set current HP to max HP at start

GRAPH **TakeDamage function — full flowchart**

Create a new Function named `TakeDamage` with input **DamageInfo (FS_DamageInfo)**.

TAKEDAMAGE FUNCTION GRAPH

Function Entry
- DamageInfo
- ◀

Input: the damage payload struct

↓

Branch
- Condition ← bIsInvincible
- ◀ True → do nothing
- ◀ False →

Skip damage if player is invincible

False pin continues

↓

Float − Float
- A ← CurrentHealth
- B ← DamageInfo.BaseDamage
- Result → ●

Subtract damage from current HP

↓

Clamp (Float)
- Value ← result above
- Min = 0.0
- Max ← MaxHealth
- Return → ●

Health can never go below 0 or above max

↓

SET CurrentHealth
- Value ← clamped result
- ◀

Store the new HP value

↓

Call OnDamageTaken
- DamageAmount ← DamageInfo.BaseDamage
- ◀

Broadcast — HUD, effects etc. listen to this

↓

Branch — is dead?
- Condition: CurrentHealth ≤ 0
- ◀ True →
- ◀ False → end

Check if HP reached zero

True pin

↓

Call OnDeath
- ◀ Exec Out

Broadcast death — game mode, enemy, UI all react

FLOW SUMMARY — AC_HEALTHCOMPONENT

1. **BeginPlay:** CurrentHealth is set to MaxHealth so the player starts at full HP.
2. **TakeDamage called:** First checks bIsInvincible — if true, the whole function exits immediately with no damage applied.
3. **Damage calculation:** Subtracts BaseDamage from CurrentHealth and clamps the result between 0 and MaxHealth. Health can never go negative.
4. **Broadcast:** OnDamageTaken fires with the damage amount — the HUD binds to this to update the health bar without the component knowing about the UI.
5. **Death check:** If CurrentHealth hit 0, OnDeath fires — the Game Mode or enemy manager listens for this to trigger death sequences.

🔧 DEBUG & TEST — AC_HEALTHCOMPONENT

HOW TO TEST

- **Print health on damage:** In AC_HealthComponent, bind a `Print String` to `OnDamageTaken`. Print `"HP: " + CurrentHealth` in cyan. Fire at yourself to confirm it decrements.
- **Test invincibility:** In the Details panel on the placed player, set `bIsInvincible=true` at runtime (use the Outliner → Details during PIE). Take damage — health should not change. Flip it back to false and confirm damage resumes.
- **Force death:** In BeginPlay (temporarily), call `TakeDamage` with BaseDamage=9999. Confirm `OnDeath` fires — add a `Print String "DEAD"` to that event to verify.
- **Clamp check:** Take damage equal to MaxHealth + 100. CurrentHealth should stop at 0.0, never go negative.

EXPECTED RESULTS

- ✓ PASS — Each shot reduces health by exactly BaseDamage
- ✓ PASS — bIsInvincible=true: health never changes regardless of damage input
- ✓ PASS — Massive damage: health clamps to 0.0, OnDeath fires once
- ✗ FAIL — Health goes negative: Clamp node is missing or Min is not 0. Check the Clamp node in TakeDamage.
- ✗ FAIL — OnDeath fires every frame: you have a looping check. Death should only fire once — add a `bIsDead` bool gate.

AC_RAGECOMPONENT

> The heart of the combat rhythm. Owns the meter, manages the 1.5s Enraged State, and broadcasts two key events that every combat system listens to.

VARIABLES Variables + Event Dispatchers

NAME	TYPE	DEFAULT
RageMax	Float	100.0
RageCurrent	Float	0.0
EnragedDuration	Float	1.5
bIsEnraged	Bool	false
EnragedTimerHandle	FTimerHandle	—

Add 2 Event Dispatchers: `OnEnragedStateBegin` and `OnEnragedStateEnd` (both no inputs).

GRAPH AddRage function (input: Amount Float)

ADDRAGE — CALLED BY WEAPON HIT DETECTION

GRAPH

ConsumeRage function — triggers the Enraged State

CONSUMERAGE — CALLED WHEN METER FILLS

SET bIsEnraged = true

SET RageCurrent = 0.0

Reset the meter immediately

Call OnEnragedStateBegin

All listeners fire simultaneously — weapon boost, dash unlock, parry widen, skill enable

Set Timer by Function Name

- FunctionName: "EndEnragedState"
- Time ← EnragedDuration (1.5)
- Looping: false
- → SET EnragedTimerHandle

Start the 1.5s countdown. Store the handle in case we need to cancel it later.

GRAPH

EndEnragedState — Custom Event (timer callback)

ENDENRAGEDSTATE CUSTOM EVENT — FIRED BY TIMER AFTER 1.5S

Custom Event: EndEnragedState

Right-click graph → Add Custom Event → name exactly "EndEnragedState"

SET bIsEnraged = false

Call OnEnragedStateEnd

All listeners fire — dash locked, parry window reset, skill disabled

FLOW SUMMARY — AC_RAGECOMPONENT

1. **AddRage called:** Checks bIsEnraged first — rage can't accumulate during an active Enraged State. Then adds Amount to RageCurrent and clamps it.
2. **Meter full:** ConsumeRage fires. bIsEnraged becomes true, meter resets to 0, and OnEnragedStateBegin broadcasts to every bound listener simultaneously.
3. **Timer starts:** A 1.5-second one-shot timer is set. The function name "EndEnragedState" is the callback — UE will call that Custom Event when the timer expires.
4. **State ends:** bIsEnraged flips to false. OnEnragedStateEnd broadcasts — all systems that enabled special behaviour during the state now revert.
5. **Nothing is coupled:** This component never touches WeaponComponent, ParryComponent, or SkillSlotComponent directly. They all listen to its events and manage themselves.

🔧 DEBUG & TEST — A

HOW TO TEST

- **Visualise rage meter:** Add a Print String
- **Force-trigger enrage:** Temporarily call
- **Time the 1.5s window:** Note the ti when the en state start
- **Test rage block during enrage:** Shoot should during

AC_WEAPONCOMPONENT + BP_WEAPONBASE

> BP_WeaponBase is the abstract parent for all three weapons. AC_WeaponComponent manages the three-slot inventory and delegates firing to whichever weapon is active.

CREATE BP_WeaponBase — set as Abstract

In `Weapons/Base/` → Blueprint Class → parent = **Actor**. Name `BP_WeaponBase`. Open → Class Settings → check **Abstract**. Add variables: `CurrentAmmo` (Int), `MaxAmmo` (Int), `bIsReloading` (Bool), `DamageMultiplier` (Float, default 1.0). Add Event Dispatchers: `OnFired`, `OnReloadComplete`, `OnAmmoEmpty`. Add a Custom Event named `Fire` — child BPs override this.

GRAPH AC_WeaponComponent — BeginPlay: spawn all weapons

BEGINPLAY — SPAWN 3 WEAPON ACTORS AND STORE IN ARRAY

Event BeginPlay
◀

Spawn Actor: BP_Weapon_Pistol
● Owner ← GetOwner
Return Value → ●

Spawn each weapon at world origin — they're invisible until the player holds them

Array ADD → WeaponSlots
● New Item ← spawned Pistol

Slot 0 = Pistol. Repeat Spawn + ADD for Shotgun (slot 1) and AssaultRifle (slot 2).

GRAPH AC_WeaponComponent — Fire() function

FIRE — DELEGATES TO ACTIVE WEAPON, APPLIES RAGE BOOST IF ACTIVE

GET WeaponSlots[ActiveWeaponIndex]
Weapon Ref → ●

Get current weapon from array

Branch — Is Valid?
● Is Valid (weapon ref)
◀ Valid →

Always check array results before using them

Branch — bNextShotBoosted?
● bNextShotBoosted
◀ True →
◀ False →

Check if rage boost is active

TRUE — BOOSTED SHOT

SET DamageMultiplier = 3.0 on weapon

SET bNextShotBoosted = false

Call Fire on weapon ref

One 300% shot, then boost consumed

FALSE — NORMAL SHOT

Call Fire on weapon ref

Normal damage, no boost

FLOW SUMMARY — AC_WEAPONCOMPONENT

1. **BeginPlay:** All three weapons are spawned as Actors and stored in an array. Index 0 = Pistol, 1 = Shotgun, 2 = Assault Rifle.
2. **Fire() called:** Gets the weapon at ActiveWeaponIndex from the array. Validates it (never blindly use array results without checking).
3. **Rage boost check:** If bNextShotBoosted is true (set by OnEnragedStateBegin), the weapon's DamageMultiplier is set to 3.0 before firing, then immediately reset to 1.0 — the boost is consumed on that one shot.
4. **Delegation:** This component never contains shooting logic itself — it just calls Fire() on the active weapon BP. Each weapon handles its own hitscan/projectile/spread logic.

🔧 DEBUG & TEST — AC_WEAPONCOMPONENT + WEAPONS

HOW TO TEST

- **Verify weapon spawning:** In BP_PlayerCharacter BeginPlay, after WeaponComponent initialises, print `WeaponSlots.Num()`. Should be 3. If less, one of the Spawn Actor calls failed.
- **Test weapon switching:** Call `SwitchWeapon(1)` via a debug key. Print `ActiveWeaponIndex` — should be 1. Fire — should behave like Shotgun.
- **Test rage boost shot:** Manually set `bNextShotBoosted=true` via the Details panel during PIE. Fire once. Confirm damage is 3× (print DamageInfo.BaseDamage in the enemy's ReceiveDamage). bNextShotBoosted should be false after one shot.
- **Draw debug traces:** In each weapon's Fire(), add a `Draw Debug Line` node alongside the Line Trace — same Start/End. Red lines appear in the viewport showing every shot's path.

EXPECTED RESULTS

- ✓ PASS — WeaponSlots.Num() = 3 at BeginPlay
- ✓ PASS — SwitchWeapon(2): AR fire behaviour, slowed movement on hold
- ✓ PASS — Boosted shot deals exactly 3× normal damage
- ✓ PASS — Debug lines visible in viewport on each shot
- ✗ FAIL — Boost applies to every shot: bNextShotBoosted is not being reset to false after firing. Check the SET false node runs before the weapon's Fire() call.

GRAPH **ActivateParry function**

ACTIVATEPARRY — CALLED ON PLAYER INPUT

Branch — NOT bParryActive
- NOT bParryActive
- ◀ True → can activate

Prevent double-activation

↓

SET bParryActive = true
- ◀

↓

Spawn Actor BP_ParryWindow
- Location ← GetOwner Location
- → SET ParryWindowHandle

Activate the overlap hitbox on the player

↓

Set Timer → "DeactivateParry"
- Time ← CurrentParryWindow
- Looping: false

Auto-close parry window after the window duration expires

GRAPH **BP_ParryWindow — On Overlap with projectile**

BP_PARRYWINDOW SPHERE COLLISION — ON COMPONENT BEGIN OVERLAP

On Component Begin Overlap
- Other Actor → ●

↓

CanBeParried (Interface)
- ● Target ← Other Actor
- Return → ●

> Ask the projectile: are you currently parryable?

↓

Branch
- ● Condition ← CanBeParried result
- ◀ True →

↓

Branch — Perfect timing?
- ● Age ≤ PerfectParryWindow (e.g. 0.08s)
- ◀ True →
- ◀ False →

> Compare GetGameTimeSinceCreation against window age

PERFECT

OnPerfectParried (Interface)
- ● Target ← projectile

> Triggers stagger on source enemy via AC_StunComponent

NORMAL

OnParried (Interface)
- ● Target ← projectile

> Projectile handles redirection or destruction

FLOW SUMMARY — AC_PARRYCOMPONENT

1. **Player presses parry:** ActivateParry checks it isn't already active, spawns BP_ParryWindow (a sphere collision around the player), and starts a timer to close it after CurrentParryWindow seconds.
2. **Projectile enters sphere:** BP_ParryWindow asks the projectile "CanBeParried?" via interface. Only highlighted (green) projectiles return true.
3. **Timing check:** The age of the parry window is compared against a tight perfect-parry threshold (~80ms). Perfect = stagger on the enemy via OnPerfectParried. Normal = projectile deflection via OnParried.
4. **Rage state widens the window:** OnEnragedStateBegin (bound in BP_PlayerCharacter BeginPlay) calls SetWindowLeniency(2.0) — doubling CurrentParryWindow. OnEnragedStateEnd resets it to 1.0.

🔧 DEBUG & TEST — AC_PARRYCOMPONENT

HOW TO TEST

- **Visualise parry sphere:** In BP_ParryWindow, add a `Draw Debug Sphere` node on BeginPlay. Set radius=80, colour=yellow, duration=CurrentParryWindow. Press parry and see the sphere appear around the player.
- **Test parry on live projectile:** Place a FodderRanged enemy. Press parry when a highlighted (green) projectile is incoming. Confirm it's destroyed and Pistol gains ammo.
- **Test perfect parry timing:** Add a `Print String "PERFECT"` to BP_ParryWindow's OnPerfectParried branch. Time your parry to fire in the first 80ms of the window. The print should appear and the enemy should stagger.
- **Test window during enrage:** Trigger enrage, activate parry. The yellow debug sphere should persist for twice as long as normal (CurrentParryWindow × 2).

EXPECTED RESULTS

- ✓ PASS — Debug sphere visible for exactly CurrentParryWindow seconds
- ✓ PASS — Green projectile + parry input: projectile destroyed, ammo +1
- ✓ PASS — "PERFECT" prints and enemy staggers on tight timing
- ✗ FAIL — Parry triggers on grey projectile: CanBeParried is returning true when it shouldn't. Check bCanBeParried is false before OpenParryWindow fires.
- ✗ FAIL — Window never closes: DeactivateParry timer isn't firing. Check the function name string matches exactly.

GRAPH TryGrapple — latch onto anchor and pull player

TRYGRAPPLE FUNCTION

Line Trace By Channel

- Start ← Camera Location
- End ← Camera Forward × GrappleRange
- Hit Actor →

Shoot a ray forward to find a grapple anchor

Cast to BP_GrappleAnchor

- Object ← Hit Actor
- Success →
- Fail → exit

Only valid if it's a designated grapple anchor

SET CurrentAnchor

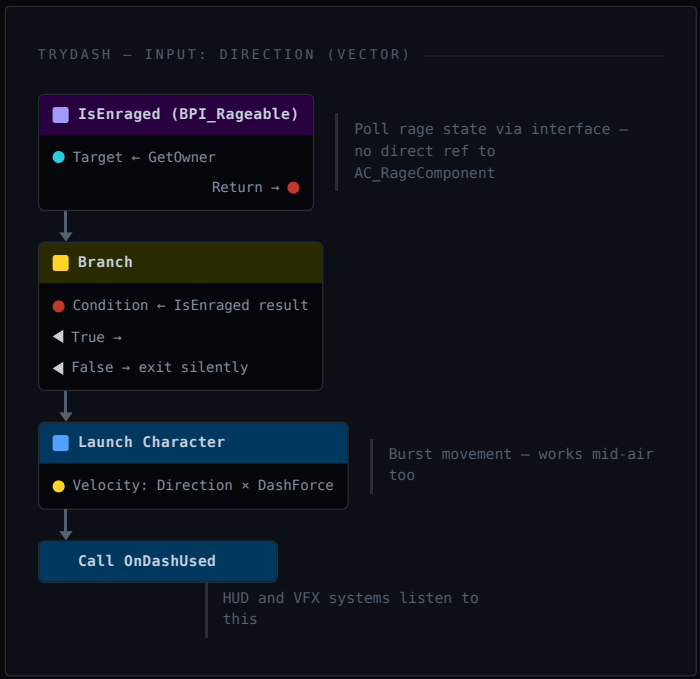
- Value ← cast result

Launch Character

- Velocity: (AnchorLoc − PlayerLoc).Normalize
- bXYOverride: true
- bZOverride: true

Calculate direction to anchor and launch player toward it

GRAPH **TryDash — rage-gated directional burst**



FLOW SUMMARY — AC_MOVEMENTEXTCOMPONENT

1. **Grapple:** A line trace from the camera finds a BP_GrappleAnchor. If hit, the player is launched toward it using Character Movement's LaunchCharacter — no physics cable needed, just a velocity impulse.
2. **Dash gate:** TryDash calls IsEnraged on the owner via BPI_Rageable. If false, the function exits silently — no dash, no error. This is the interface pattern keeping systems decoupled.
3. **Dash execution:** LaunchCharacter with the input direction × DashForce. Setting bXYOverride and bZOverride true means the impulse fully overrides current velocity for that burst.

🔧 DEBUG & TEST — AC_MOVEMENTEXTCOMPONENT

HOW TO TEST

- **Test grapple trace:** Add `Draw Debug Line` (green, 2s duration) in TryGrapple from Camera Location to hit location. Fire the grapple — a green line should appear showing exactly where the trace went.
- **Test grapple on non-anchor:** Aim at a wall (no BP_GrappleAnchor). Press grapple. The Cast to BP_GrappleAnchor should fail silently — no movement, no crash.
- **Test dash gate:** Try dashing without being enraged. Nothing should happen. Print `"Dash attempted, enraged: " + IsEnraged result` to confirm the gate is working.
- **Test dash mid-air:** Jump, then trigger enrage (debug shortcut), then dash. Player should burst in the input direction regardless of being airborne.

EXPECTED RESULTS

- ✓ PASS — Grapple trace: green line visible in viewport, player launches toward anchor
- ✓ PASS — Grapple on wall: no movement, no error in Output Log
- ✓ PASS — Dash outside enrage: no movement
- ✓ PASS — Dash during enrage: burst in correct direction, works mid-air
- ✗ FAIL — Grapple launches in wrong direction: check the Normalize node is applied to (AnchorLoc − PlayerLoc) before multiplying by GrappleSpeed.

GRAPH **ThrowSatchel + BP_SatchelCharge detonation**

FLOW SUMMARY — AC_UTILITYCOMPONENT + BP_SATCHELCHARGE

1. **ThrowSatchel:** Gates on bCanThrow, spawns BP_SatchelCharge, adds a physics impulse for the throw arc, then starts the 45s cooldown immediately.
2. **Fuse:** The moment BP_SatchelCharge spawns, BeginPlay starts a 2-second timer. The "Detonate" Custom Event fires when it expires.
3. **Detonation:** FireImpulse on the RadialForceComponent pushes all nearby physics objects — including the player for a blast jump. Then GetOverlappingActors loops through everything in radius and calls ReceiveDamage on each via BPI_Damageable.
4. **Self-contained:** The satchel knows nothing about the player or enemies specifically. It just deals damage to whatever implements BPI_Damageable.

🔧 DEBUG & TEST — AC_UTILITYCOMPONENT + SATCHEL

HOW TO TEST

- **Visualise detonation radius:** In BP_SatchelCharge's Detonate event, add `Draw Debug Sphere` at the satchel's location with radius = ShockwaveRadius, colour=orange, duration=2s. Confirm it matches the damage area.
- **Test blast jump:** Throw the satchel, stand directly on top of it, let it detonate. The radial force should launch the player upward. If it doesn't, check `bImpulseVelChange=true` on the RadialForceComponent and verify ShockwaveForce is large enough (try 800000).
- **Test cooldown gate:** Throw once — immediately try to throw again. bCanThrow should be false. Print it on the input event to confirm. Wait 45s (or temporarily set Cooldown=3s) and confirm the second throw works.
- **Test damage in radius:** Place a dummy enemy inside the debug sphere radius. Detonate. Its health should decrease. Place one outside — it should be unaffected.

EXPECTED RESULTS

- ✓ PASS — Orange sphere visible at detonation, matches damaged area
- ✓ PASS — Player standing on satchel: launched upward on detonation
- ✓ PASS — Second throw immediately: blocked (bCanThrow=false)
- ✓ PASS — Enemy inside radius takes damage, enemy outside does not
- ✗ FAIL — Satchel detonates immediately on spawn: DetonationDelay timer is 0 or negative. Check the Time pin on the Set Timer node.

BP_PLAYERCHARACTER — FULL WIRING

> The Player Character is the hub. Add all components in the Components panel, implement the two interfaces, and bind all rage event listeners in BeginPlay.

SETUP Add components in the Components panel

Open BP_PlayerCharacter → in the **Components panel** click **+ Add** and search each class by name. Rename each after adding:

- AC_HealthComponent → `HealthComponent`
- AC_RageComponent → `RageComponent`
- AC_WeaponComponent → `WeaponComponent`
- AC_MovementExtComponent → `MovementExtComponent`
- AC_ParryComponent → `ParryComponent`
- AC_UtilityComponent → `UtilityComponent`
- AC_SkillSlotComponent → `SkillSlotComponent`

Then Class Settings → Implemented Interfaces → Add: `BPI_Damageable`, `BPI_Rageable`.

GRAPH BeginPlay — bind all rage listeners

Use a **Sequence node** off BeginPlay to chain all bindings. A Sequence fires all its outputs in order — cleaner than one long chain.

BEGINPLAY RAGE BINDINGS — REPEAT THIS PATTERN FOR EACH LISTENER

Repeat this **Bind → Custom Event → Action** pattern for all rage listeners:

DISPATCHER	CUSTOM EVENT NAME	WHAT IT DOES
OnEnragedStateBegin	OnRage_WeaponBoost	WeaponComponent: SET bNextShotBoosted = true
OnEnragedStateBegin	OnRage_UnlockDash	MovementExtComponent: SET bDashAvailable = true
OnEnragedStateBegin	OnRage_WidenParry	ParryComponent: SetWindowLeniency(2.0)
OnEnragedStateBegin	OnRage_EnableSkill	SkillSlotComponent: SET bSkillAvailable = true
OnEnragedStateEnd	OnRageEnd_LockDash	MovementExtComponent: SET bDashAvailable = false
OnEnragedStateEnd	OnRageEnd_ResetParry	ParryComponent: SetWindowLeniency(1.0)
OnEnragedStateEnd	OnRageEnd_DisableSkill	SkillSlotComponent: SET bSkillAvailable = false

FLOW SUMMARY — BP_PLAYERCHARACTER BEGINPLAY

1. **Components are added via the Components panel** — not spawned in BeginPlay. UE creates them automatically when the Actor spawns.
2. **Interfaces implemented:** BPI_Damageable routes ReceiveDamage to HealthComponent.TakeDamage. BPI_Rageable.NotifyWeakPointHit routes to RageComponent.AddRage. IsEnraged just reads and returns RageComponent.bIsEnraged.
3. **Rage bindings in BeginPlay:** Each binding connects a Custom Event to one of the two rage dispatchers. When the dispatcher fires, all bound events fire simultaneously — no ordering issues, fully decoupled.
4. **Input routing:** In the Event Graph, Enhanced Input events (IA_Fire, IA_Parry, IA_Grapple etc.) just call the matching function on the relevant component. The character BP is a thin router.

🔧 DEBUG & TEST — BP_PLAYERCHARACTER FULL WIRING

HOW TO TEST

- **Verify all components exist:** During PIE, select the player in the Outliner → Details panel → scroll to Components. All 7 AC_ components should be listed. If any is missing, the + Add step was skipped.
- **Test each rage binding:** Trigger enrage (debug shortcut). Check each system individually: fire one shot (should be 3×), try to dash (should work), parry a projectile (wider window), activate skill (should work). All four should respond.
- **Test interface implementations:** Call `IsEnraged` and `GetCurrentHealth` on the player from a separate test BP using interface calls. Confirm both return valid values.
- **Test input routing:** Add a `Print String` at the start of every input event (IA_Fire, IA_Parry, etc.) to confirm each one fires when you press the corresponding input.

EXPECTED RESULTS

- ✓ PASS — Details panel shows 7 AC_ components on the player
- ✓ PASS — All 4 rage abilities active during enrage, all 3 revert on end
- ✓ PASS — Interface calls from external BP return correct values
- ✗ FAIL — Rage bindings don't fire: BeginPlay bindings might not be executing. Add a Print String right after each Bind Event node and confirm they run at game start.

DATATABLES — CREATING AND READING

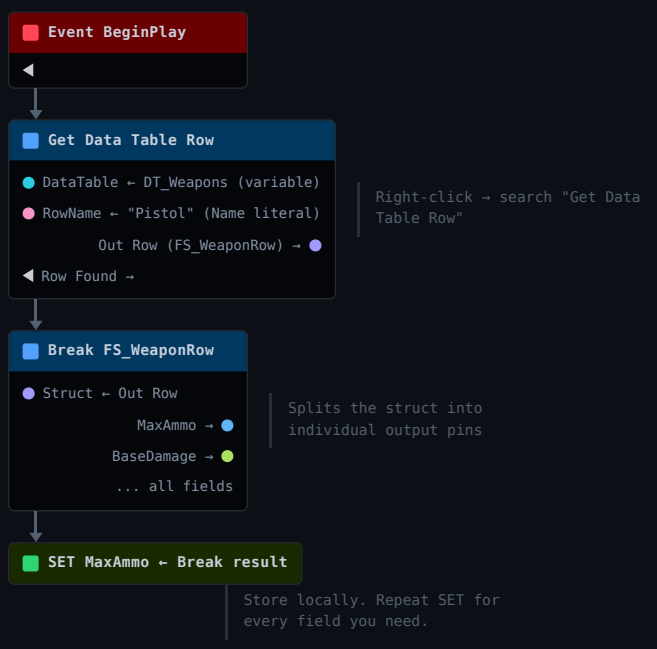
All tunable values live in DataTables. A designer changes weapon damage by editing a row — no Blueprint graph ever needs to open.

CREATE Create a DataTable

First create the struct (e.g. `FS_WeaponRow` in `Structs/` → right-click → Blueprint → Structure). Then in `Data/` → right-click → Miscellaneous → Data Table → pick your struct. Open it and click **+ Add** to create rows — name them `Pistol`, `Shotgun`, `AssaultRifle`.

GRAPH Reading a DataTable row at runtime

EXAMPLE: WEAPON BP READS ITS OWN STATS FROM DT_WEAPONS ON BEGINPLAY



FLOW SUMMARY — DATATABLES

1. **Struct defines the columns** — the Blueprint Structure asset describes what data each row contains. Create the struct first, then create the DataTable using it.
2. **Get Data Table Row** takes the table asset and a row name (a Name literal, not a String), and outputs the full row struct plus an exec pin "Row Found" that only fires if the row exists.
3. **Break Struct** splits the row output into individual pins — one per field. Connect each to a SET node to store the values as local variables.
4. **Designer workflow:** To change pistol damage, open DT_Weapons → find row "Pistol" → change BaseDamage. No Blueprint graph opened. No compile needed.

🔧 DEBUG & TEST — DATATABLES

HOW TO TEST

- **Test row not found:** Deliberately mistype a row name (e.g. "Pistoll"). The `Get Data Table Row` node's **Row Found** exec pin will not fire. Add a `Print String "ROW MISSING"` on the other exec pin to catch this.
- **Live value change:** During PIE, open DT_Weapons and change Pistol's BaseDamage to 999. Stop PIE, restart. Shoot the enemy — it should die in one hit. Confirms the table is read at runtime, not compiled in.
- **Verify all rows load:** In BP_EnemyBase BeginPlay, print the row name being queried. Each enemy type should print its own row name, confirming each child BP correctly sets its row identifier.

EXPECTED RESULTS

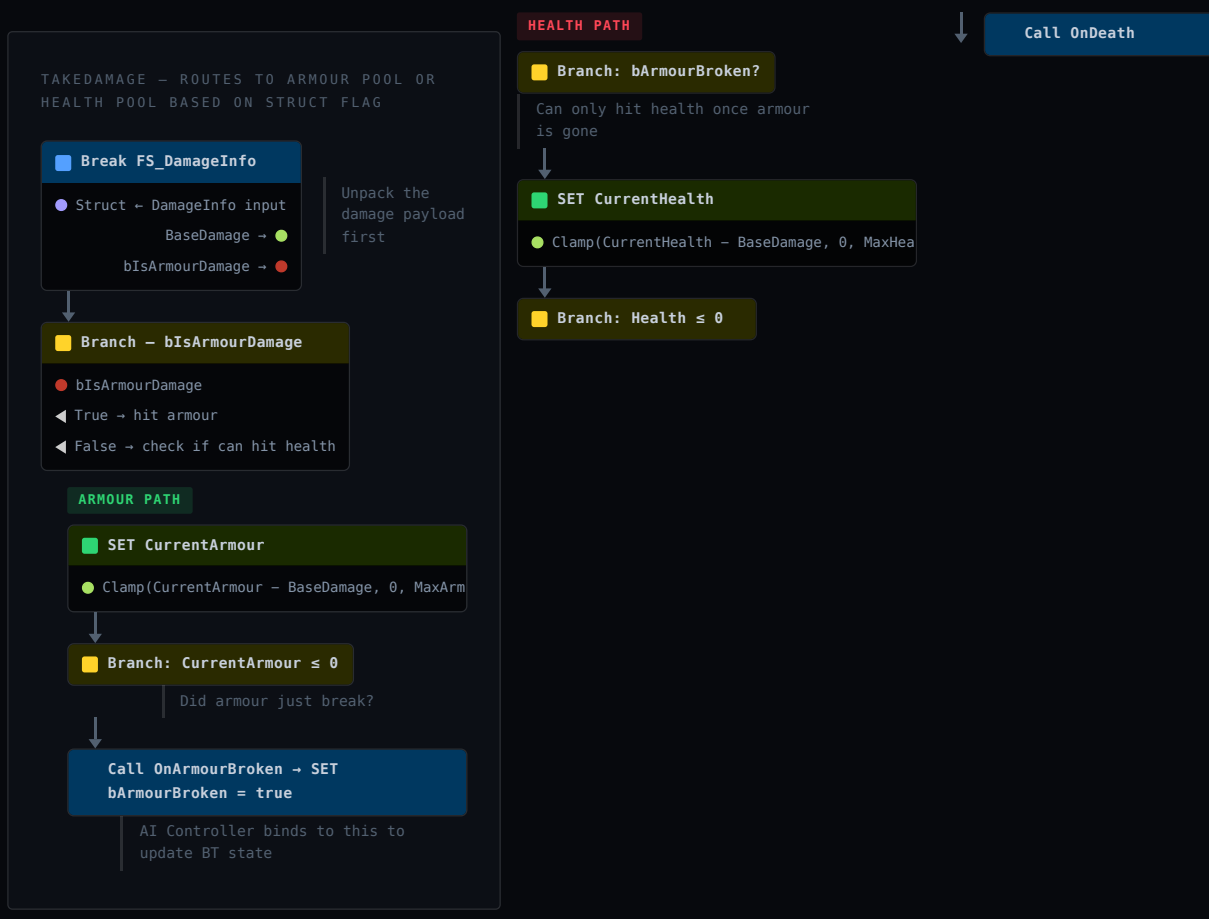
- ✓ PASS — Correct row name: Row Found fires, struct values are valid
- ✓ PASS — Wrong row name: "ROW MISSING" prints, no crash
- ✓ PASS — BaseDamage=999: enemy dies in one pistol shot
- ✗ FAIL — Struct values are all zero after break: Row Found exec isn't connected. The Break node only runs if Row Found fires — make sure exec wires are connected, not just data wires.

BP_ENEMYBASE + AC_ENEMYHEALTHCOMPONENT

SETUP Create BP_EnemyBase (Abstract)

In `Enemies/Base/` → Blueprint Class → parent = **Character**. Name `BP_EnemyBase`. Open → Class Settings → check **Abstract**. Implement: `BPI_Damageable`, `BPI_WeakPointTarget`. Add components: `AC_EnemyHealthComponent`, `AC_WeakPointComponent`, `AC_StunComponent`, plus UE native **AIPerception** (click + Add in Components panel → search "AI Perception").

GRAPH AC_EnemyHealthComponent — TakeDamage with armour routing



FLOW SUMMARY — AC_ENEMYHEALTHCOMPONENT

1. **Armour routing:** The bIsArmourDamage flag in the struct decides which pool takes the damage. This lets the Assault Rifle deal armour damage (bIsArmourDamage=true) while the Pistol deals health damage (bIsArmourDamage=false, bypasses armour entirely on weak points).
2. **Armour breaking:** When CurrentArmour reaches 0, OnArmourBroken fires. The AIC_ binds to this in BeginPlay and writes bArmourBroken=true to the Blackboard — unlocking the Brute's exposed weak point attack phase.
3. **Health protection:** The health branch checks bArmourBroken first. Damage can only reach the health pool once the enemy's armour has been stripped.
4. **Death:** OnDeath fires when health hits 0. The ArenaManager and AC_SpawnDirectorComponent (on the Spawner) bind to enemy OnDeath to track wave completion and refill spawn budgets.

🔧 DEBUG & TEST — AC_ENEMYHEALTHCOMPONENT

HOW TO TEST

- **Print both pools:** In TakeDamage, after each SET node, print `"Armour: " + CurrentArmour + " HP: " + CurrentHealth`. Fire AR at Brute (armour damage) — armour decreases, health stays. Shoot exposed WP (health damage) — health decreases.
- **Test armour break event:** Add `Print String "ARMOUR BROKEN"` to OnArmourBroken. Strip the Brute's armour completely — the print should appear exactly once.
- **Test health gate:** Temporarily set bArmourBroken=false during PIE. Try to deal health damage. Health should not change — armour must be broken first.
- **Force death:** Call TakeDamage with BaseDamage=9999 on an enemy with no armour. OnDeath should fire — add a print to confirm.

EXPECTED RESULTS

- ✓ PASS — AR vs armoured Brute: armour decreases, health=100 unchanged
- ✓ PASS — Armour reaches 0: "ARMOUR BROKEN" prints once, bArmourBroken=true
- ✓ PASS — Health damage before armour broken: health unchanged
- ✗ FAIL — Health drops while armour > 0: the bArmourBroken branch check is missing. Add it before the health subtraction.
- ✗ FAIL — OnDeath fires multiple times: add a `bIsDead` bool gate, same as AC_HealthComponent.

AC_WEAKPOINTCOMPONENT + BP_WEAKPOINTACTOR

SETUP

Create BP_WeakPointActor

In `Combat/` → Blueprint Class → parent = Actor. Name `BP_WeakPointActor`. Add a **Sphere Collision** component (Radius = 30). Add variables: `WeakPointIndex` (Int), `bIsExposed` (Bool, default true — set false for armoured WPs like the Brute).

GRAPH

BP_WeakPointActor — On Hit detection

ON COMPONENT HIT — FIRES WHEN A BULLET/TRACE HITS THE WP COLLISION SPHERE

On Component Hit
- Other Actor → ●

Bind this to the Sphere Collision component in BeginPlay

↓

Branch — bIsExposed
- ● bIsExposed
- ◀ True → WP is hittable
- ◀ False → exit, armour blocks

Brute WPs start with bIsExposed=false until armour breaks

↓

RegisterWeakPointHit (BPI_WeakPointTarget)
- ● Target ← GetOwner (the enemy)
- ● WeakPointIndex
- ● DamageDealt

Tell the enemy which WP was hit

↓

NotifyWeakPointHit (BPI_Rageable)
- ● Target ← Get Player Character (index 0)
- ● RageAmount ← from DA_EnemyData

Feed rage to the player — WP actor doesn't need to know about AC_RageComponent

GRAPH

AC_WeakPointComponent — spawn WP actors at BeginPlay

BEGINPLAY — CREATE ONE PER SOCKET AND ATTACH I

For Each Loop — WPSocke
- Array Element (soc
- Ar
- ◀ Loop Body →

↓

Spawn Actor BP_WeakPoi
- Spawned Ac

↓

Attach Actor to Compone
- ● Actor ← spawned WP
- ● Parent ← GetOwner Mesh
- ● Socket Name ← loop elemen

↓

SET WeakPointIndex on W Array Index

Each WP knows its number for report the enemy

FLOW SUMMARY — AC_WEAKPOINTCOMPONENT

1. **Spawn on BeginPlay:** The component loops through WPSocketNames (define and spawns one BP_WeakPointActor per socket, atta enemy's skeletal mesh so WPs move with the animat
2. **Hit detection:** BP_WeakPointActor's sphere collision fires On Com player projectile or trace hits it.
3. **Armour gate:** bIsExposed blocks hit registration. For the Brute, AC_EnemyHealthComponent sets this to true on its WP OnArmourBroken fires.
4. **Dual notification:** On a valid WP hit, two interface calls go out (RegisterWeakPointHit) and one to the player (Neither the WP actor nor the component needs a either BP class.

🔧 DEBUG & TEST — AC_WEAKPOINTCOMPONENT

HOW TO TEST

- **Visualise WP spheres:** In BP_WeakPointActor BeginPlay, add `Draw Debug Sphere` at its own location, radius=30, colour=red, duration=9999. In PIE you should see red spheres floating at each socket position, moving with the enemy's animations.
- **Test hit notification:** Add `Print String "WP HIT: " + WeakPointIndex` inside the On Component Hit event before the interface calls. Shoot a WP — the index should print to confirm which WP was hit.
- **Test rage generation:** Watch the rage meter (or print RageCurrent) while hitting WPs. Each confirmed hit should add RageAmountPerHit to the meter.
- **Test armour block on Brute:** Set bIsExposed=false on a WP actor via the Details panel during PIE. Shoot it — the hit notification should NOT print (execution stops at the Branch).

EXPECTED RE

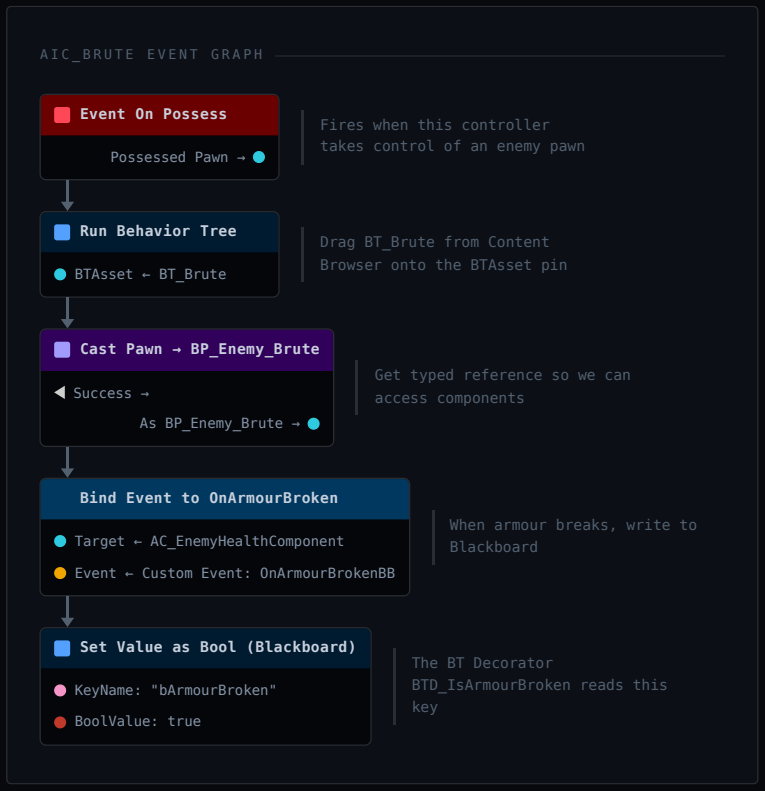
- ✓ PASS — Red correc animat
- ✓ PASS — "WP etc. p
- ✓ PASS — Rag each
- ✓ PASS — bIsE notifi genera
- ✗ FAIL — WP s in wro socket WPSock the ac the en asset.

AI CONTROLLER SETUP + BLACKBOARDS

The AI Controller connects the Pawn to the Behavior Tree and handles perception — writing detected actors to the Blackboard so the BT can act on them.

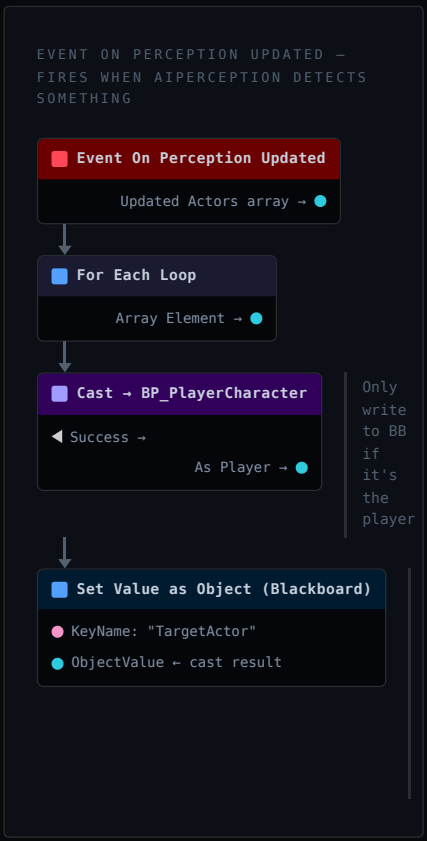
GRAPH

AIC_Brute — OnPossess starts the BT, BeginPlay binds component events



GRAPH

AIC — OnPerceptionUpdated: write player to Blackboard



💡 **Assign the AIC to the enemy:** Open BP_Enemy_Brute → Details panel → Pawn → **AI Controller Class** = AIC_Brute. Without this assignment UE uses the default AIController and your BT will never run.

SETUP

Blackboard keys for all three main enemies

In each enemy's folder → right-click → AI → Blackboard. Open it and click **New Key** for each entry below. Then open the matching Behavior Tree and assign this Blackboard in the Details panel.

BB_	BB_	BB_
KEY	KEY	KEY
Targ	Tar	Tar
Targ	Str	Ret
bHas	bHa	Spa
bIsS	bEy	bCa
bArm	bIm	bIsI
Atta	Fir	

FLOW SUMMARY — AI CONTROLLER + BLACKBOARDS

1. **OnPossess:** The AIC immediately calls Run Behavior Tree. The BT won't run without this. It also casts the possessed pawn to its specific enemy class so it can access components and bind to their events.
2. **Perception → Blackboard:** OnPerceptionUpdated loops through detected actors, finds the player via Cast, and writes the player reference to the TargetActor key. BT movement and attack tasks then read this key.
3. **Component events → Blackboard:** The AIC binds to component dispatchers (OnArmourBroken, OnStunBegin, OnStunEnd) and writes the matching Blackboard booleans. The BT's Decorators read these booleans to gate branches.
4. **The Blackboard is the bridge:** Nothing in the BT talks directly to a Blueprint component. It only reads and writes keys. This means you can completely redesign a component's internals and the BT never needs to change — only the AIC bindings need updating.

🔧 DEBUG & TEST — AI CONTROLLER + BLACKBOARDS

HOW TO TEST	EXPECTED RESULTS
Open Blackboard Debugger: During PIE, select any enemy → Window → Developer Tools → Blackboard. You'll see all keys and their live values updating in real time as the AI runs. Essential for verifying key writes.	✓ PASS — BB Debugger: TargetActor populates on player entry to sight range
Test perception: Walk into the enemy's sight radius. Watch the BB Debugger — TargetActor should populate with your player reference. Walk away beyond LoseSightRadius — it should clear (or stay as last known).	✓ PASS — bArmourBroken flips true in debugger when armour = 0
Test ArmourBroken write: Strip the Brute's armour in PIE. In the BB Debugger, bArmourBroken should flip from false to true the moment armour hits 0.	✓ PASS — bIsStunned flips true in debugger during sustained fire
Test AIC assignment: If the enemy stands still and does nothing after spawning, the AI Controller is likely not assigned. Check BP_Enemy_X → Class Defaults → Pawn → AI Controller Class.	✗ FAIL — TargetActor never populates: AIPerceptionComponent config may be wrong. Check SightConfig is added and the Dominant Sense is set to Sight. Also ensure the player has an AIPerceptionStimuliSource component.
	✗ FAIL — Enemy stands still: no AIC assigned, or Run Behavior Tree isn't called in OnPossess.

BEHAVIOR TREES

Behavior Trees are built visually in UE's BT editor — no Blueprint graph here. Create them in the enemy folder: right-click → AI → Behavior Tree. Assign the matching Blackboard in the Details panel (top right).

BT How to read the tree tables below

Indent level shows parent-child relationship. A **Selector** tries children left-to-right and stops at the first success. A **Sequence** runs all children in order and stops at the first failure. **Decorators** (marked D:) gate whether a branch can run. **Services** (marked S:) tick while their parent node is active.

💡 **Adding Decorators and Services:** Right-click any Selector or Sequence node in the BT editor → Add Decorator or Add Service → pick from the list. They appear as small tags attached to the top of the node.

⚠️ **Always call Finish Execute in custom BT Tasks.** Every BT Task you create must call the **Finish Execute** node (with Success=true or false) when it's done. If you forget this, the Behavior Tree freezes permanently on that task — the most common BT bug beginners hit.

BT

BT_Brute — full tree layout

NODE	TYPE / CONFIG	NOTES
Root	—	Always present, one child only
└ Selector	Main	Tries branches left to right
├ Sequence	D: BTD_IsStunned (Inverse=false)	Only runs WHILE stunned — Inverse=false means "condition must be true to enter"
│ └ Wait	Duration=0.5s	Brute stands still during stun
├ Sequence	Combat — S: BTS_UpdatePlayerLocation	Service ticks player position every 0.2s
│ ├ BTT_FindPlayerLocation	Task — writes TargetLocation key	
│ ├ Move To	Key=TargetLocation, AcceptRadius=150	Move into melee range
│ └ Selector	Attack choice	Picks attack based on armour state
│ ├ BTT_AttackMelee	D: BTD_IsArmourBroken (true)	High-damage WP strike when armour is gone
│ └ BTT_AttackMelee	No decorator	Standard strike — deals armour damage
└ BTT_FindPlayerLocation	Fallback	If combat sequence fails, keep moving toward last known location

BT

BT_FlyingEye — full tree layout

NODE	TYPE / CONFIG	NOTES
Root → Selector	Main	
├ Sequence	D: BB Key bImmobilised = true	Eye WP destroyed — only this branch runs
│ └ Wait	Duration=9999	Effectively disables the enemy
├ Sequence	Strafe+Fire — S: BTS_UpdatePlayerLocation, S: BTS_CheckLineOfSight	Both services tick while this branch is active
│ ├ Run EQS Query	Query=EQS_FlyingEye_Strafe → writes StrafeDestination	Find a good lateral position
│ ├ Move To	Key=StrafeDestination	Fly to strafe position
│ └ Sequence	D: BB Key bHasLOS = true	Only fire if the player is visible
│ └ BTT_FireProjectile	Target=TargetActor	Spawns parryable projectile toward player
└ BTT_FindPlayerLocation	Fallback	Move toward last known player position

📌 **EQS for FlyingEye strafe:** Right-click in enemy folder → AI → Environment Query. Generator = **Points: Circle** centred on the Querier's target. Radius=400, NumPoints=16. Add a Distance test to the Player → Score so points 300–500 units away score highest. The Run EQS Query task writes the winning location to StrafeDestination.

BT

BT_Spawner — full tree layout

NODE	TYPE / CONFIG	NOTES
Root → Sequence	Main — S: BTS_UpdatePlayerLocation	Always tracking player
├ Run EQS Query	Query=EQS_Spawner_Retreat → writes RetreatDestination	Find farthest navmesh point from player
├ Move To	Key=RetreatDestination	Flee
└ Selector	Spawn Decision	
├ BTT_SpawnFodder	D: BB Key bCanSpawn = true	Only spawn when budget remains
└ Wait	Duration=2.0	Pause before re-evaluating

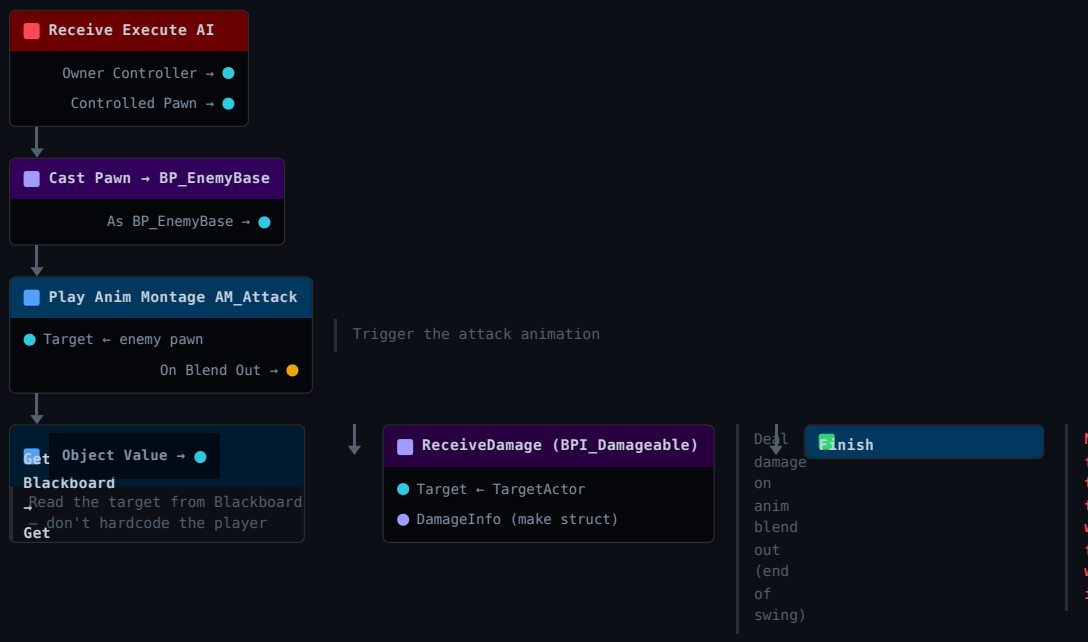
📌 **EQS for Spawner retreat:** Generator = Points: Grid, GridHalfSize=1000, SpaceBetween=200. Add a Distance test to the Player → Filter out points closer than 600 units → Score remaining by maximum distance. This picks the farthest valid navmesh point.

BT TASK

BTT_AttackMelee — how to build a custom BT Task

Right-click in `Enemies/Shared_AI/` → AI → Behavior Tree Task. Name `BTT_AttackMelee`. Open it. Wire **Receive Execute AI**:

BTT_ATTACKMELEE — RECEIVE EXECUTE AI



FLOW SUMMARY — BEHAVIOR TREES

1. **Selector vs Sequence:** Selectors try children until one succeeds (OR logic). Sequences run children until one fails (AND logic). Understanding this difference is the key to reading any BT.
2. **Decorators gate branches:** BTD_IsStunned returns the bIsStunned Blackboard value. When true, the attack Sequence is blocked. When false, it can run. No Blueprint logic needed — pure data.
3. **Services tick while active:** BTS_UpdatePlayerLocation runs every 0.2s while the Combat Sequence is active. If the Sequence fails and the fallback branch runs, the service stops ticking automatically.
4. **Tasks do one thing:** Each BTT_ performs a single action and calls Finish Execute when done. This composability is what makes the BT easy to extend — add a new task, drop it into any tree.
5. **EQS provides spatial intelligence:** Instead of hardcoding "stay 400 units away", the FlyingEye runs an EQS query that finds valid positions in the game world dynamically, accounting for obstacles and navmesh.

🔧 DEBUG & TEST — BEHAVIOR TREES

HOW TO TEST

- **BT Visual Debugger:** Window → Developer Tools → Behavior Tree Debugger. Select an enemy during PIE. The active node highlights green. Watch the tree execute in real time — the most powerful BT debug tool available.
- **Test decorator gating:** While the Brute is stunned, the stunned Sequence should show green. The combat Sequence should be grey (blocked). Trigger stun end — it should swap.
- **Test EQS visually:** Enable EQS debugging: in Project Settings → AI → check `Allow EQS Debug Drawing`. Press apostrophe key (') during PIE to toggle. Coloured spheres appear showing all query points scored by the EQS.
- **Test Finish Execute:** If a custom task causes the BT to freeze entirely (enemy stops all behaviour), a Finish Execute is missing. Use the BT Debugger to see which task is stuck (stays green indefinitely).

EXPECTED RESULTS

- ✓ PASS — BT Debugger: active node changes as enemy moves through states
- ✓ PASS — Stun: Stunned branch green, Combat branch grey
- ✓ PASS — EQS debug: coloured score spheres visible around player
- ✗ FAIL — BT freezes on a task: open BT Debugger, find the green node that never changes, go to that task BP and verify Finish Execute is connected to all code paths including failure.
- ✗ FAIL — Enemy ignores player: NavMesh may not cover the floor. Press P to visualise it. A green overlay must appear under the enemy for pathfinding to work.